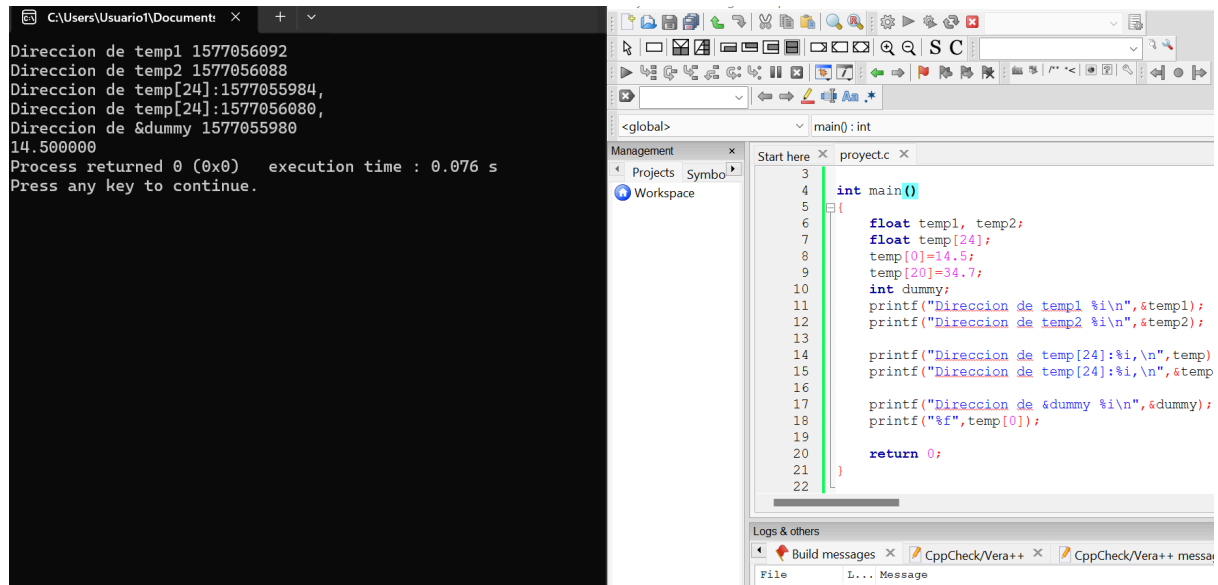


Direcciones de memoria en arreglo

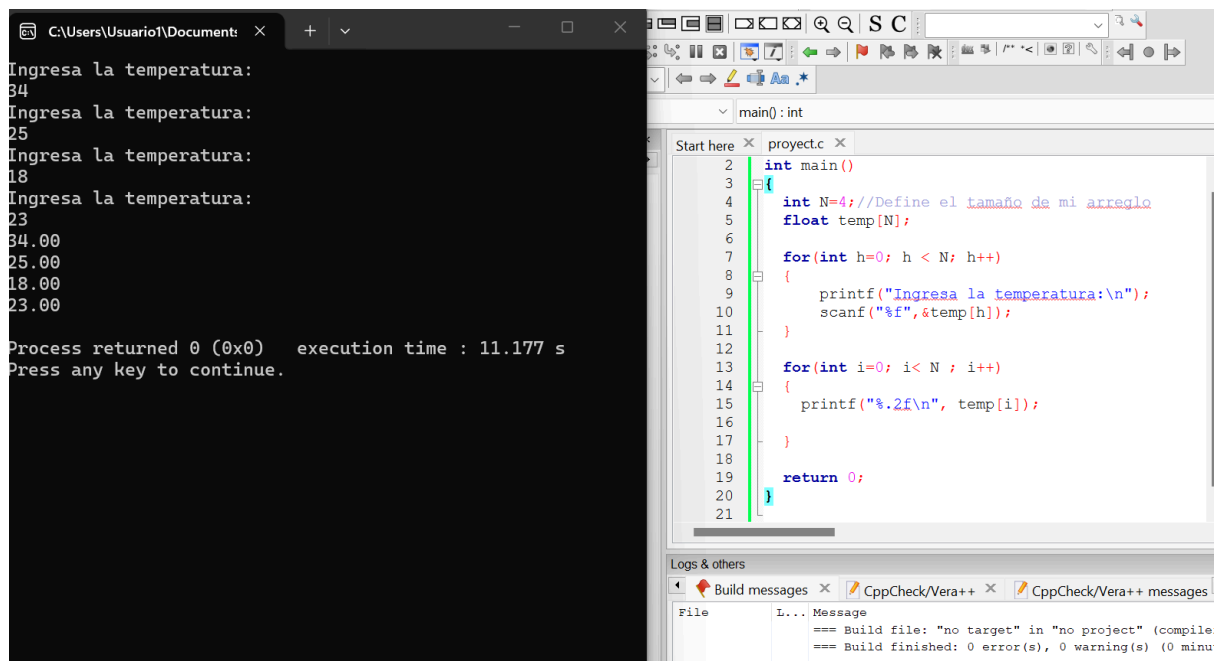


The screenshot shows a C++ IDE with a terminal window on the left and a code editor on the right. The terminal displays the memory addresses of variables: `Direccion de temp1 1577056092`, `Direccion de temp2 1577056088`, `Direccion de temp[24]:1577055984,`, `Direccion de &dummy 1577055980`, and `14.500000`. The code editor shows the following C++ code:

```
3
4 int main()
5 {
6     float temp1, temp2;
7     float temp[24];
8     temp[0]=14.5;
9     temp[20]=34.7;
10    int dummy;
11    printf("Direccion de temp1 %i\n",&temp1);
12    printf("Direccion de temp2 %i\n",&temp2);
13
14    printf("Direccion de temp[24]:%i,\n",temp)
15    printf("Direccion de temp[24]:%i,\n",&temp)
16
17    printf("Direccion de &dummy %i\n",&dummy);
18    printf("%f",temp[0]);
19
20    return 0;
21 }
```

The Logs & others window at the bottom shows build messages and CppCheck/Vera++ messages.

Temperaturas

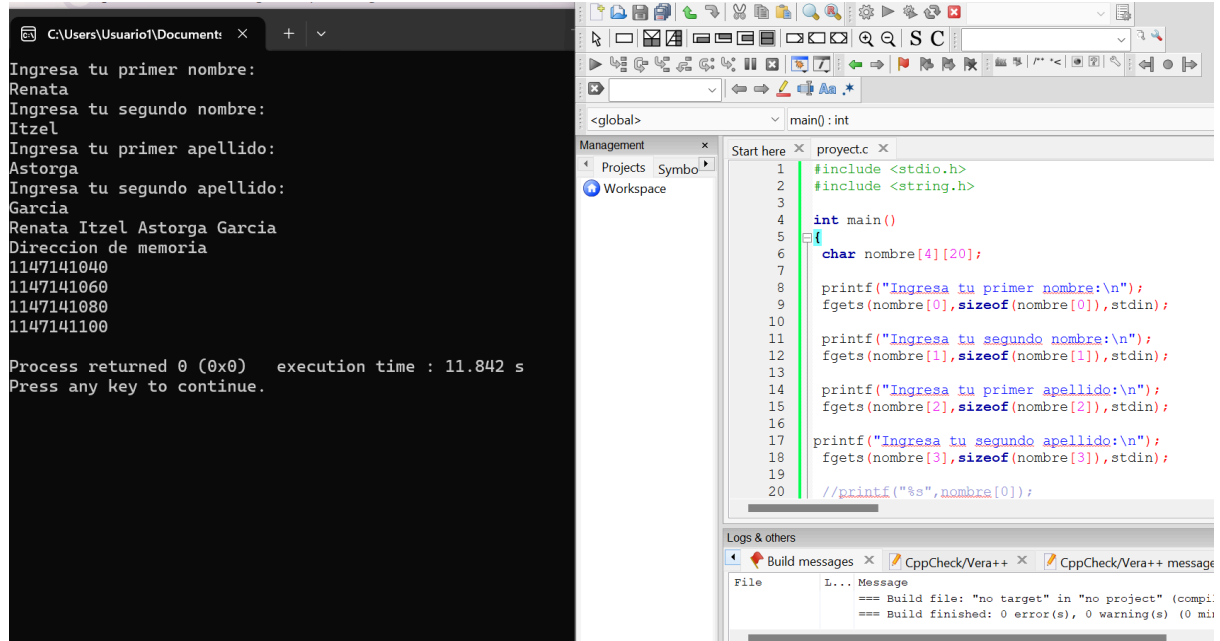


The screenshot shows a C++ IDE with a terminal window on the left and a code editor on the right. The terminal displays the input and output of the program: `Ingresa la temperatura:`, `34`, `Ingresa la temperatura:`, `25`, `Ingresa la temperatura:`, `18`, `Ingresa la temperatura:`, `23`, `34.00`, `25.00`, `18.00`, `23.00`, and `Process returned 0 (0x0) execution time : 11.177 s`. The code editor shows the following C++ code:

```
2 int main()
3 {
4     int N=4;//Define el tamaño de mi arreglo
5     float temp[N];
6
7     for(int h=0; h < N; h++)
8     {
9         printf("Ingresa la temperatura:\n");
10        scanf("%f",&temp[h]);
11    }
12
13    for(int i=0; i < N ; i++)
14    {
15        printf("%.2f\n", temp[i]);
16    }
17
18    return 0;
19 }
```

The Logs & others window at the bottom shows build messages and CppCheck/Vera++ messages.

Nombre completo



The image shows a C program for collecting a full name. On the left, a terminal window displays the program's execution: it prompts for a first name (Renata), a second name (Itzel), a first surname (Astorga), and a second surname (Garcia), then prints the full name and memory addresses. On the right, a code editor shows the source code in 'proyect.c', which uses `printf` and `fgets` to handle input and `sizeof` for array sizes. A build log at the bottom indicates a successful compilation.

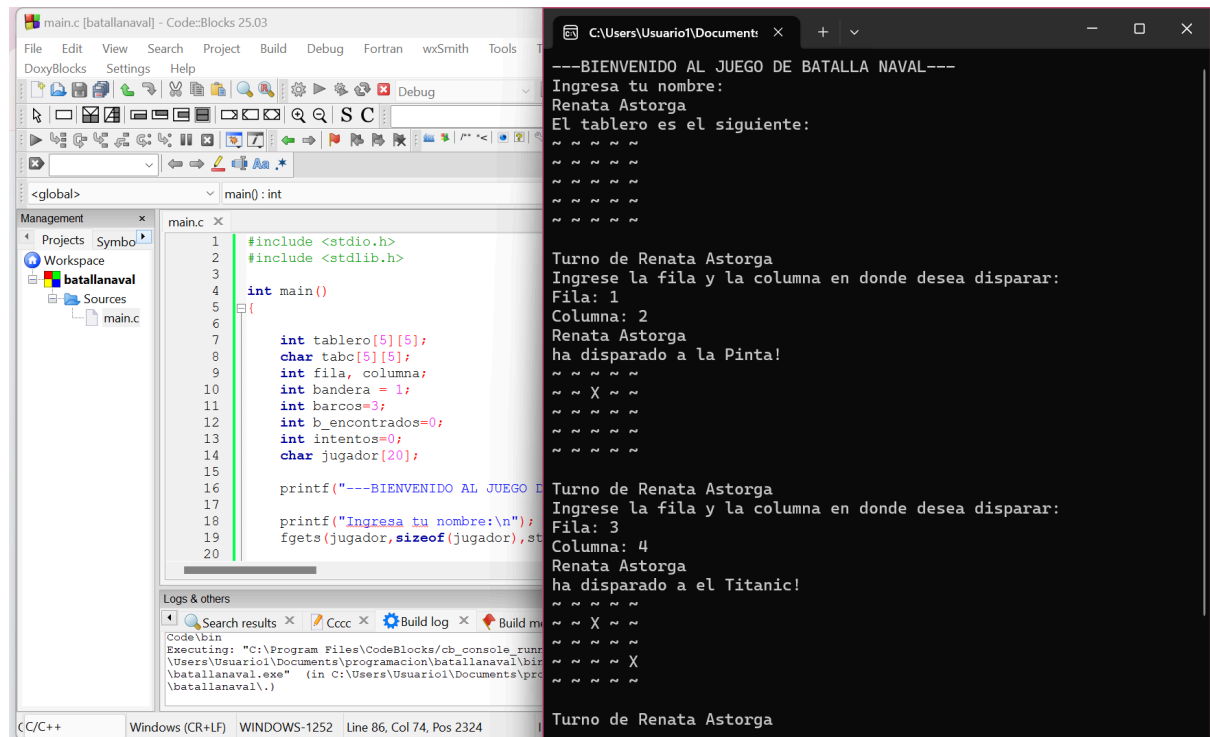
```
Ingresa tu primer nombre:
Renata
Ingresa tu segundo nombre:
Itzel
Ingresa tu primer apellido:
Astorga
Ingresa tu segundo apellido:
Garcia
Renata Itzel Astorga Garcia
Direccion de memoria
1147141040
1147141060
1147141080
1147141100

Process returned 0 (0x0)   execution time : 11.842 s
Press any key to continue.
```

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char nombre[4][20];
7
8     printf("Ingresa tu primer nombre:\n");
9     fgets(nombre[0], sizeof(nombre[0]), stdin);
10
11    printf("Ingresa tu segundo nombre:\n");
12    fgets(nombre[1], sizeof(nombre[1]), stdin);
13
14    printf("Ingresa tu primer apellido:\n");
15    fgets(nombre[2], sizeof(nombre[2]), stdin);
16
17    printf("Ingresa tu segundo apellido:\n");
18    fgets(nombre[3], sizeof(nombre[3]), stdin);
19
20    //printf("%s", nombre[0]);
```

Build messages: Build file: "no target" in "no project" (compil... Build finished: 0 error(s), 0 warning(s) (0 mi...

Batalla Naval



The image shows a C program for a naval battle game. On the left, a code editor displays the source code in 'main.c', which includes `stdio.h` and `stdlib.h`. It defines a 5x5 grid, a player name, and variables for ship counts and attempts. On the right, a terminal window shows the game's execution: it welcomes the player (Renata Astorga), displays the empty 5x5 grid, and prompts for coordinates to shoot. The first shot at (1,2) hits a ship (Pinta), and the second shot at (3,4) hits the Titanic. The build log at the bottom shows the program was compiled successfully.

```
---BIENVENIDO AL JUEGO DE BATALLA NAVAL---
Ingresa tu nombre:
Renata Astorga
El tablero es el siguiente:
~ ~ ~ ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~

Turno de Renata Astorga
Ingresa la fila y la columna en donde desea disparar:
Fila: 1
Columna: 2
Renata Astorga
ha disparado a la Pinta!
~ ~ ~ ~ ~
~ ~ X ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~

Turno de Renata Astorga
Ingresa la fila y la columna en donde desea disparar:
Fila: 3
Columna: 4
Renata Astorga
ha disparado a el Titanic!
~ ~ ~ ~ ~
~ ~ X ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~
~ ~ ~ ~ ~

Turno de Renata Astorga
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6
7     int tablero[5][5];
8     char tabc[5][5];
9     int fila, columna;
10    int bandera = 1;
11    int barcos=3;
12    int b_encontrados=0;
13    int intentos=0;
14    char jugador[20];
15
16    printf("---BIENVENIDO AL JUEGO DE BATALLA NAVAL---\n");
17
18    printf("Ingresa tu nombre:\n");
19    fgets(jugador, sizeof(jugador), stdin);
20
```

Search results: Cccc Build log Build m... Executing: "C:\Program Files\CodeBlocks\cb_console_run... \Users\Usuario1\Documents\programacion\batallanaval\bin\batallanaval.exe" (in C:\Users\Usuario1\Documents\pro... \batallanaval\.)

NOTAS

ProgramacionEstructure / Arreglos01 / NotasArreglo01.md

↑ Top

Preview

Code

Blame



Raw



Los arreglos son un conjunto de datos del mismo tipo.

```
float temp[24]
```



CARACTERISTICAS DEL ARREGLO -"temp" es la direccion de memoria mas conocido como la etiqueta en un arreglo,

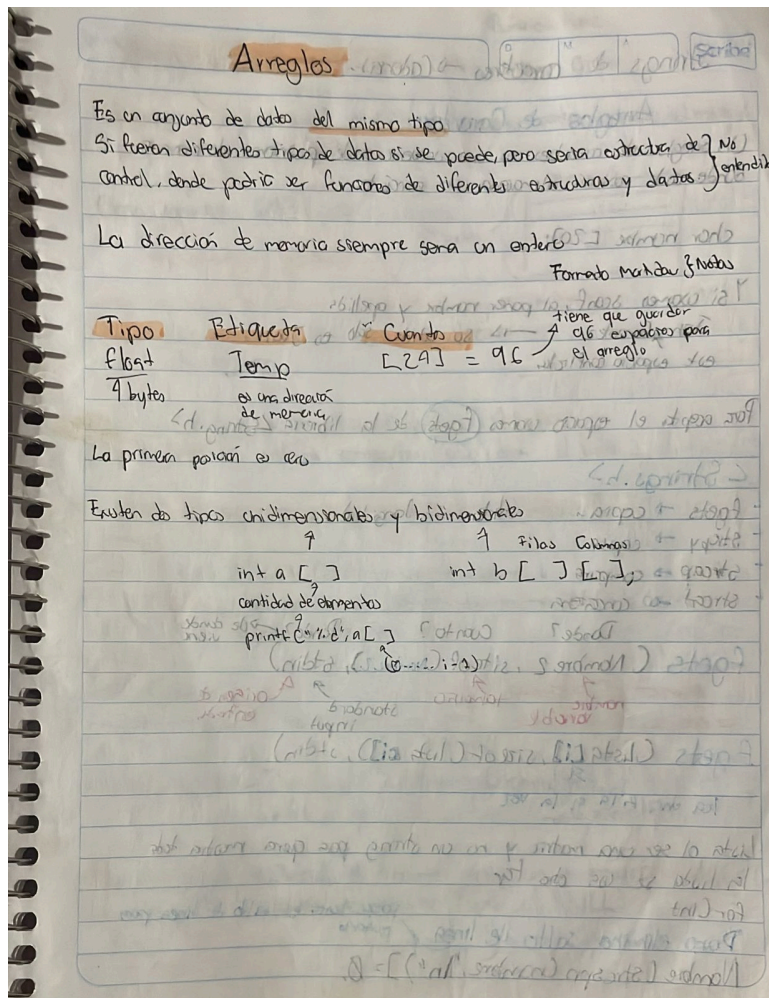
-El float es el tipo de dato.

-[24] Es la cantidad dentro del arreglo y siempre empieza en cero, es decir, que el primer valor del arreglo es 0 y el ultimo seria 23.

Cuando es un arreglo de 24 valores del mismo tipo guardo 24 espacios en la memoria(para cada valor), Por ejemplo 24 floats, cada float son 4 bytes, entonces 24x4 son 96, entonces 96 es el numero de espacios o bytes para el arreglo

Una mejor manera de manejar el valor de tu arreglo es usar una variable como una letra y la defines el valor, esto es una gran ventaja si quieres modificar el valor de tu arreglo, es decir, si quieres que sea mayor o menor la cantidad dentro del arreglo, asi ya no tienes que cambiar el valor en cada arreglo que tengas en el programa, sino que solo modificar a la variable.

```
int N=120;  
float temp[N];
```



Strings de caracteres → Cadenas

Arreglos de Caracteres

Un arreglo de caracteres es una secuencia de posiciones de memoria donde cada posición almacena un carácter.

char nombre [20];

Y si usamos scanf, al poner nombre y apellidos

Juan Gonzalez → su salida solo es Juan

este espacio conflictivo

Para aceptar el espacio usamos fgets de la librería <string.h>

Arreglo Unidimensional

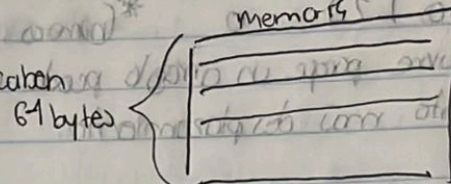
Buffer overflow

Row Major Order

Un arreglo unidimensional permite almacenar varios valores del mismo tipo usando un solo identificador.

char nombre [4] [20]

Apuntador = puntero



Contiene la dirección de memoria e indica donde empieza el arreglo.

0 = El apuntador es distinto a la dirección de memoria

int las direcciones van de 4 en 4 por sus 4 bytes

char las direcciones van en consecutivos por tener 1 byte

Dirección de memoria con un arreglo bidimensional